

# Ejercicio para llevar a casa — Construí tu propio tokenizer de GPT-4

**Inteligencia Artificial Generativa Avanzada y Sistemas Multi-Agente** Licenciatura en Tecnología Digital —  
UTDT **Clase 3 · Tokenización**

**Carácter del ejercicio:** opcional y **no evaluado**. No se entrega ni se corrige. Existe para que puedas fijar lo que vimos en clase escribiendo el código vos mismo, que es la única forma de que la tokenización deje de ser una caja negra.

## 1. Objetivo y contexto

El objetivo es que implementes, desde cero, un tokenizer BPE byte-level y que llegues a **reproducir exactamente el tokenizer real de GPT-4** (`cl100k_base`), token por token, contra la librería `tiktoken`.

Cada paso del ejercicio corresponde a una pieza que ya vimos en clase:

- **texto** → **bytes**. Vimos que la unidad base no son los ~150.000 code points de Unicode sino los **256 bytes** de UTF-8. En el Paso 1 vas a materializar eso: tu vocabulario arranca con exactamente 256 entradas y todo lo demás se construye encima.
- **BPE**. Vimos que "entrenar" un tokenizer es puro conteo de pares y no tiene nada de gradiente. En el Paso 1 vas a escribir ese loop y a producir las dos estructuras que salen de él: `merges` y `vocab`.
- **encode / decode**. Vimos que `encode` aplica los merges por **prioridad de merge** (el orden en que se crearon durante el entrenamiento) y **no** por frecuencia en el texto de entrada, y que `decode` es un simple lookup en `vocab`. En el Paso 1 vas a comprobar que el round-trip `decode(encode(s)) == s` es exacto, y a medir la compresión en bytes por token.
- **regex splitting**. Vimos *por qué* GPT-2/GPT-4 parten el texto antes de correr BPE: para que los merges no crucen límites de palabra, número y puntuación. En el Paso 2 vas a agregar ese pre-split y a ver la diferencia en el vocabulario resultante.
- **tokens especiales**. Vimos `<|endoftext|>`, el formato ChatML y el *carve-out* que se hace **antes** de BPE, más el footgun de `allowed_special`. Ese es el Paso 4.
- **SentencePiece**. Lo mencionamos al pasar: corre BPE sobre code points en vez de bytes. Ese es el Paso 5, que es de estiramiento y **no tiene solución de referencia** (ver la advertencia en ese paso).

El ejercicio está basado en el `exercise.md` del repositorio `karpathy/minbpe`, adaptado para la materia.

## 2. Prerrequisitos

Software

| Requisito | Detalle                                                                     |
|-----------|-----------------------------------------------------------------------------|
| Python    | 3.9 o superior. Verificado sobre <b>3.11.9</b> y <b>3.12.12</b> en Windows. |
| git       | Para clonar el repositorio.                                                 |

| Requisito           | Detalle                                                                                                                                            |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Conexión a internet | <b>Necesaria</b> para instalar dependencias y, la primera vez, para que <b>tiktoken</b> descargue el vocabulario de <b>cl100k_base</b> (ver §3.4). |

## Dependencias de Python

```
regex      # regex compatible con \p{L}, \p{N}; NO alcanza el módulo re de la
stdlib
tiktoken   # tokenizer oficial de OpenAI; se usa como librería local, sin API key
pytest     # para correr los tests del repositorio
```

Todas son open-source y se instalan desde PyPI, y las tres están en el **requirements.txt** del repositorio con la versión exacta fijada, para que todos trabajemos sobre lo mismo. **tiktoken no requiere API key ni cuenta de OpenAI**: se usa puramente como librería local, para (a) comparar tu salida contra la referencia y (b) leer los merges de GPT-4 en el Paso 3.

## 3. Setup y verificación inicial

### 3.1 Clonar el repositorio

```
git clone https://github.com/solidgoldmagickarp/tokenizer.git
cd tokenizer
```

### 3.2 Crear un entorno virtual e instalar dependencias

**Usá siempre un entorno virtual**, no instales en el Python del sistema.

```
# Windows (PowerShell)
python -m venv .venv
.venv\Scripts\Activate.ps1

# Windows (Git Bash)
python -m venv .venv
source .venv/Scripts/activate

# Linux / macOS
python3 -m venv .venv
source .venv/bin/activate
```

Con el entorno activo:

```
pip install -r requirements.txt
```

Verificá que el entorno virtual está realmente activo antes de instalar:

```
python -c "import sys; print(sys.prefix)"  
# tiene que imprimir una ruta que termine en ...\tokenizer\.venv
```

### 3.3 Confirmar que el entorno está listo

```
python verificar_setup.py
```

Chequea cuatro cosas y las reporta una por una: versión de Python, dependencias instaladas, el corpus de entrenamiento, y el vocabulario de GPT-4 (que descarga si hace falta, ver §3.4). Si algo sale **FALLA**, es un problema de instalación y hay que resolverlo antes de escribir una sola línea de código.

Después probá los tests:

```
pytest -v .
```

Esperado: **21 tests, todos en FAILED**, en menos de un segundo. **Eso está bien**. Los esqueletos de `mi_tokenizer/` levantan `NotImplementedError` a propósito: ese es el estado inicial. Tu trabajo durante todo el ejercicio es ir pasándolos a verde, un paso por vez:

```
pytest -m paso1 -v      # los 5 tests del Paso 1  
pytest -m paso2 -v      # los 6 del Paso 2  
pytest -m paso3 -v      # los 8 del Paso 3  
pytest -m paso4 -v      # los 2 del Paso 4
```

### 3.4 Primera ejecución de `tiktoken`: descarga del vocabulario

La primera vez que corras algo que use `tiktoken.get_encoding("cl100k_base")` (`verificar_setup.py` o los tests, por ejemplo), la librería **descarga** el archivo de vocabulario de GPT-4 desde [https://openaipublic.blob.core.windows.net/encodings/cl100k\\_base.tiktoken](https://openaipublic.blob.core.windows.net/encodings/cl100k_base.tiktoken) (~1,7 MB) y lo cachea en disco.

Consecuencias prácticas:

- **Necesitás internet la primera vez.** Después funciona offline.
- Por defecto la caché va al directorio temporal del sistema (`%TEMP%\data-gym-cache` en Windows, `/tmp/data-gym-cache` en Linux/macOS), que **se puede borrar solo**. Si querés que la caché sea persistente, definí la variable de entorno `TIKTOKEN_CACHE_DIR` apuntando a una carpeta tuya:

```
# Windows (PowerShell)
$env:TIKTOKEN_CACHE_DIR = "$PWD\.tiktoken_cache"

# Linux / macOS
export TIKTOKEN_CACHE_DIR="$PWD/.tiktoken_cache"
```

- Si estás detrás de un proxy corporativo que bloquea ese dominio, la descarga falla y **no vas a poder hacer los Pasos 3 y 4**. Hacé esta prueba antes de sentarte a trabajar sin conexión.

### 3.5 Mirá adónde querés llegar

Antes de escribir código, conviene ver qué produce un tokenizer entrenado. En la carpeta `ejemplos/` están las salidas reales de entrenar con `vocab_size=512` sobre `tests/taylorswift.txt` (el artículo de Wikipedia sobre Taylor Swift, ~186 KB):

```
ejemplos/basic.vocab    <- BasicTokenizer  (Paso 1)
ejemplos/regex.vocab   <- RegexTokenizer (Paso 2)
```

Abrilos con un editor de texto. Cada línea es un merge, en el orden en que se creó: `[e][r] -> [er]` 256. Son la mejor forma de entender qué aprende el algoritmo, y compararlos entre sí ya te adelanta buena parte del Paso 2.

Una vez que termines el Paso 2, vas a poder generar tus propias versiones de esos archivos:

```
python train.py      # tarda ~25 s; escribe models/basic.* y models/regex.*
```

- `.model` es el archivo que se vuelve a cargar con `load()`.
- `.vocab` es **solo para leer**: usa `errors="replace"` y por lo tanto es lossy. Nunca se lee de vuelta.

Si querés ver `train.py` corriendo de entrada, podés hacerlo desde la rama con la solución (ver §9).

### 3.6 Tu espacio de trabajo

Ya está armado. El repositorio trae los esqueletos y la suite de tests:

```
mi_tokenizer/
  __init__.py
  base.py      <- clase base. Casi entera; faltan get_stats y merge (Paso 1)
  basic.py     <- Paso 1
  regex.py     <- Pasos 2 y 4
  gpt4.py      <- Paso 3
tests/
  test_mi_tokenizer.py <- los 21 tests que validan tu código
  taylorswift.txt     <- el corpus
```

Cada método que tenés que escribir está como stub: la firma, un docstring con el contrato y una guía de implementación, y un `raise NotImplementedError`. Buscá los `NotImplementedError` y andá reemplazándolos.

**No hace falta que respetes esta estructura** si preferís otra: lo único que los tests necesitan es que `mi_tokenizer` exporte `BasicTokenizer`, `RegexTokenizer` y `GPT4Tokenizer` con la interfaz descripta. Pero si la cambiás, acordate de que los tests tienen que seguir importando.

## 4. Los pasos del ejercicio

Son cinco pasos, en orden de dificultad creciente. Cada uno se apoya en el anterior.

| Paso | Qué construís                                          | Carácter                                          |
|------|--------------------------------------------------------|---------------------------------------------------|
| 1    | <code>BasicTokenizer</code> — BPE sobre bytes          | <b>Obligatorio</b>                                |
| 2    | <code>RegexTokenizer</code> — agrega el split de GPT-4 | <b>Obligatorio</b>                                |
| 3    | Reproducir GPT-4 ( <code>cl100k_base</code> )          | <b>Obligatorio</b> (es el objetivo del ejercicio) |
| 4    | Tokens especiales                                      | Opcional                                          |
| 5    | SentencePiece / Llama 2                                | Stretch, sin solución de referencia               |

### Paso 1 — `BasicTokenizer`

**Qué:** BPE directamente sobre los bytes UTF-8 del texto. Sin regex splitting, sin tokens especiales. Es la versión mínima correcta del algoritmo.

**Dónde:** `mi_tokenizer/basic.py` (la clase `BasicTokenizer`) y las funciones `get_stats` / `merge` de `mi_tokenizer/base.py`. **Autochequeo continuo:** `pytest -m paso1 -v` (5 tests).

#### Funciones a implementar:

```
class BasicTokenizer:
    def train(self, text, vocab_size, verbose=False):
        """Entrena merges y vocab a partir de text. vocab_size >= 256."""

    def encode(self, text):
        """str -> list[int]"""

    def decode(self, ids):
        """list[int] -> str"""
```

#### Qué se espera:

- El vocabulario arranca siempre con los **256 bytes** (`vocab = {i: bytes([i]) for i in range(256)}`), y se hacen `vocab_size - 256` merges.
- `train` mantiene dos estructuras: `merges: dict[(int, int), int]` y `vocab: dict[int, bytes]`. El `vocab` se **deriva** de los merges (`vocab[idx] = vocab[p0] + vocab[p1]`), nunca se escribe a mano.

- En cada iteración: contás todos los pares consecutivos, tomás el más frecuente, le asignás el siguiente id libre ( $256 + i$ ) y lo reemplazás en la secuencia.
- **encode no** usa frecuencias: en cada vuelta busca, entre los pares presentes en la secuencia actual, el que tenga el **menor índice de merge** (es decir, el que se creó primero durante el entrenamiento), y lo aplica. Repite hasta que no quede ningún par que esté en **merges**.
- **decode** concatena `vocab[idx]` para cada id y hace `.decode("utf-8", errors="replace")`.

**Autochequeo A — el ejemplo canónico de la clase.** Es el mismo que vimos:

```
tokenizer = BasicTokenizer()
text = "aaabdaaabc"
tokenizer.train(text, 256 + 3)      # 256 bytes + 3 merges
print(tokenizer.encode(text))
# [258, 100, 258, 97, 99]
print(tokenizer.decode([258, 100, 258, 97, 99]))
# aaabdaaabc
```

Recordá que `a=97`, `b=98`, `c=99`, `d=100` (ASCII), así que los ids nuevos arrancan en 256.

**Ojo con un detalle que confunde.** Wikipedia describe el resultado como `X=ZY`, `Y=ab`, `Z=aa`. Si mirás el **vocab** que produce esta implementación vas a ver otra cosa:

```
256 -> b'aa'
257 -> b'aaa'
258 -> b'aaab'
```

No es un error. En el segundo merge hay un **empate**: tanto `(256, 97)` como `(97, 98)` aparecen 2 veces, y `max()` en Python se queda con el primero que encuentra. Las dos formas de romper el empate llevan al **mismo resultado final** `[258, 100, 258, 97, 99]`, por caminos distintos. Chequeá los ids finales, no el árbol de merges intermedio.

**Autochequeo B — entrenar sobre un corpus real.**

```
text = open("tests/taylorswift.txt", "r", encoding="utf-8").read()
tokenizer = BasicTokenizer()
tokenizer.train(text, 512, verbose=True)
```

Mirá los merges que salen. Los primeros deberían ser fragmentos frecuentes del inglés y basura de Wikipedia: `'e '`, `' '`, `'d '`, `'20'`, `'in'`, `'on'`, `'the '`, `'ing'`, etcétera. Si te salen cosas que no parecen fragmentos de palabras, algo anda mal.

Medí la compresión:

```
ids = tokenizer.encode(text)
nbytes = len(text.encode("utf-8"))
```

```
print(f"{nbytes} bytes -> {len(ids)} tokens = {nbytes/len(ids):.2f}X")
```

Valores de referencia con este mismo corpus y `vocab_size=512`: **185.768 bytes** → **78.746 tokens** ≈ **2,36X**.

**Advertencia de performance:** `encode` sobre los 186 KB completos tarda unos **6-7 segundos** con esta implementación (recuenta las estadísticas de toda la secuencia en cada iteración). Es esperable: el repositorio prioriza claridad sobre velocidad. Si estás iterando, probá primero con un fragmento.

## Paso 2 — `RegexTokenizer`

**Qué:** convertir tu `BasicTokenizer` en un `RegexTokenizer` que primero parte el texto con el patrón de GPT-4, procesa cada trozo por separado, y recién después concatena.

**Dónde:** `mi_tokenizer/regex.py` (la clase `RegexTokenizer`). **Autochequeo continuo:** `pytest -m paso2 -v` (6 tests).

**El patrón:**

```
GPT4_SPLIT_PATTERN = r"""'(?i:[sdmt]|ll|ve|re)|[\r\n\p{L}\p{N}]?+\p{L}+|\p{N}{1,3}| ?[\s\p{L}\p{N}]+[\r\n]*|\s*[\r\n]|\s+(?!\S)|\s+"""
```

```
import regex as re          # la librería `regex`, NO el `re` de la stdlib
compiled = re.compile(GPT4_SPLIT_PATTERN)
chunks = re.findall(compiled, text)
```

`\p{L}` y `\p{N}` (categorías Unicode) y los cuantificadores posesivos `++` / `?+` **no existen** en el módulo `re` de la stdlib. Por eso `regex` está en `requirements.txt`.

**Qué cambia respecto del Paso 1:**

- En `train`, en vez de una sola lista de ids tenés **una lista de listas** (una por chunk). Contás pares acumulando sobre todos los chunks, pero **nunca** contás un par que cruce el límite entre dos chunks. Ese es todo el punto del ejercicio.
- En `encode`, aplicás BPE a cada chunk por separado y concatenás los resultados.
- `decode` no cambia.

**Autochequeo — comparar los vocabularios.** Reentrená con `vocab_size=512` sobre el mismo `tests/taylorswift.txt` y compará contra el Paso 1:

```
def cruzan_categorias(vocab):
    """Cuenta tokens que mezclan letras / dígitos / espacios / puntuación."""
    n = 0
    for idx, tok in vocab.items():
        if idx < 256:
            continue
```

```
s = tok.decode("utf-8", errors="replace")
cats = set()
for ch in s:
    if ch.isalpha(): cats.add("L")
    elif ch.isdigit(): cats.add("N")
    elif ch.isspace(): cats.add("S")
    else: cats.add("P")
if len(cats) > 1:
    n += 1
return n
```

Con `BasicTokenizer` vas a contar del orden de **110** tokens mezclados: 'e ', ', ', 'the ', ', ', 20'. Con `RegexTokenizer` la cuenta baja a un puñado.

**Importante — el criterio "no hay merges entre categorías" no es literal.** El patrón de GPT-4 permite deliberadamente tres cosas que este contador va a marcar como "mezcladas":

1. **Un espacio al principio** de una palabra o de un signo (' the', ' (', ' "') — lo permite la rama `[^\r\n\p{L}\p{N}]?\p{L}+` y la rama `?[^\s\p{L}\p{N}]{1,}`.
2. **Contracciones** del inglés: 's, 'll, 've, 're, 'd, 'm, 't — primera rama del patrón.
3. **Puntuación seguida de salto de línea** (. \n, ] \n) — lo permite `[\r\n]*`.

Lo que **no** tenés que ver más son merges como 'the ' (letras + espacio a la derecha), 'e ', ', ', 20' (puntuación + espacio + dígitos). Ese es el chequeo real.

**Otra observación contraintuitiva:** con el mismo `vocab_size=512`, el `RegexTokenizer` **comprime peor** que el `BasicTokenizer` ( $\approx 2,13X$  contra  $\approx 2,36X$  sobre este corpus). Es correcto y esperable: le estás prohibiendo merges que sí reducían la cuenta de tokens. Se paga esa compresión a cambio de un vocabulario mucho más limpio y con mejores propiedades a escala. En compensación, `encode` se vuelve **mucho** más rápido ( $\approx 0,4$  s contra  $\approx 6,5$  s sobre los 186 KB), porque BPE corre sobre chunks cortos en lugar de sobre una secuencia gigante.

### Paso 3 — Reproducir el tokenizer de GPT-4

**Este es el paso difícil y el objetivo real del ejercicio.**

**Qué:** cargar los merges del tokenizer real de GPT-4 (`cl100k_base`) dentro de tu `RegexTokenizer`, y demostrar que tu `encode` y tu `decode` producen resultados **idénticos** a `tiktoken`.

**El objetivo a igualar:**

```
import tiktoken
enc = tiktoken.get_encoding("cl100k_base") # el tokenizer de GPT-4
ids = enc.encode("hello world!!!? (안녕하세요!) lol123 🤪")
text = enc.decode(ids) # recuperarás el mismo texto
```

**Dónde:** `mi_tokenizer/gpt4.py` (la clase `GPT4Tokenizer`, que hereda de tu `RegexTokenizer`). Es un wrapper: no se entrena, se construye a partir de datos ya existentes. **Autochequeo continuo:** `pytest -m`

`paso3 -v` (8 tests).

Te vas a topar con **dos escollos**. Los dos están explicados acá abajo y los dos están resueltos en la rama `solucion` (ver §9).

### Escollo (a): recuperar los merges crudos

`tiktoken` **no guarda el árbol de merges**. Guarda solamente el resultado final: un diccionario `enc._mergeable_ranks` de tipo `bytes -> int` que mapea cada secuencia de bytes ya fusionada a su rango (su prioridad de merge). Lo que vos necesitás es `merges: (int, int) -> int`, o sea, saber de qué par salió cada token.

La recuperación se hace corriendo un mini-BPE sobre cada token: para el token de rango `r`, se aplican todos los merges de rango **estrictamente menor** que `r`, y lo que queda tienen que ser exactamente **dos** piezas — esas dos piezas son el par que lo generó.

```
def bpe(mergeable_ranks, token, max_rank):
    parts = [bytes([b]) for b in token]
    while True:
        min_idx, min_rank = None, None
        for i, pair in enumerate(zip(parts[:-1], parts[1:])):
            rank = mergeable_ranks.get(pair[0] + pair[1])
            if rank is not None and (min_rank is None or rank < min_rank):
                min_idx, min_rank = i, rank
        if min_rank is None or (max_rank is not None and min_rank >= max_rank):
            break
        parts = parts[:min_idx] + [parts[min_idx] + parts[min_idx + 1]] +
        parts[min_idx + 2:]
    return parts

def recover_merges(mergeable_ranks):
    merges = {}
    for token, rank in mergeable_ranks.items():
        if len(token) == 1:
            continue # bytes crudos, no son merges
        pair = tuple(bpe(mergeable_ranks, token, max_rank=rank))
        assert len(pair) == 2
        merges[(mergeable_ranks[pair[0]], mergeable_ranks[pair[1]])] = rank
    return merges
```

Este enunciado te autoriza explícitamente a **copiar esta función**; no es la parte pedagógicamente interesante, y por eso está acá entera. Si querés entender por qué funciona, mirá `tiktoken#60` y `minbpe#11`. La idea corta: bajo ciertas condiciones alcanza con guardar los nodos padre y su rango, y el árbol se puede reconstruir.

Datos de referencia: `recover_merges` sobre `cl100k_base` tarda **~1 segundo** y devuelve **100.000 merges**, para un `vocab` de **100.256** entradas.

`_mergeable_ranks` es un atributo **privado** de `tiktoken` (empieza con guion bajo). Funciona hoy — verificado con `tiktoken 0.13.0` — pero es una API interna y podría cambiar en versiones futuras.

## Escollo (b): la permutación de bytes

Por razones históricas, GPT-4 **permutea** los 256 tokens de byte. El byte `0x61` ('a') no es el token 97: es el token 64. El espacio (`0x20`) no es el 32: es el 220.

La permutación está guardada justamente en los primeros 256 elementos de los mergeable ranks, así que se recupera en una línea:

```
self.byte_shuffle = {i: mergeable_ranks[bytes([i])] for i in range(256)}
self.inverse_byte_shuffle = {v: k for k, v in self.byte_shuffle.items()}
```

Y hay que aplicarla en **los dos sentidos**:

```
def _encode_chunk(self, text_bytes):
    # permutar ANTES de correr BPE
    text_bytes = bytes(self.byte_shuffle[b] for b in text_bytes)
    return super()._encode_chunk(text_bytes)

def decode(self, ids):
    text_bytes = b"".join(self.vocab[idx] for idx in ids)
    # des-permutar ANTES de decodificar UTF-8
    text_bytes = bytes(self.inverse_byte_shuffle[b] for b in text_bytes)
    return text_bytes.decode("utf-8", errors="replace")
```

Si te olvidás de la permutación, el síntoma es inconfundible: `encode` te devuelve ids que no se parecen en nada a los de `tiktoken`, y `decode` te devuelve texto ilegible o lleno de `?`.

Notá también que el `vocab` de esta clase se construye a partir de los merges recuperados **sin** des-permutar; la des-permutación se hace recién sobre los bytes finales en `decode`.

## Autochequeo:

```
import tiktoken
enc = tiktoken.get_encoding("cl100k_base")
mio = GPT4Tokenizer()

for texto in [
    "",
    "?",
    "hello world!!!! (안녕하세요!) lol123 😊",
    open("tests/taylorswift.txt", encoding="utf-8").read(),
]:
    assert mio.encode(texto) == enc.encode(texto), "encode no coincide"
    assert mio.decode(mio.encode(texto)) == texto, "round-trip roto"
print("OK")
```

Valores concretos para comparar a ojo:

```
texto = "hello123!!!? (안녕하세요!) 😊"
enc.encode(texto)
# [15339, 4513, 12340, 30, 320, 31495, 230, 75265, 243, 92245, 16715, 57037]
```

**Usá sí o sí texto con no-ASCII y emoji.** Con puro ASCII un montón de bugs quedan escondidos: los caracteres multi-byte son los que revelan si estás manejando bien los bytes.

**Nota:** el tokenizer de GPT-4 es *pretrained* y de solo lectura. En la implementación de referencia, `train()`, `save()` y `load()` de `GPT4Tokenizer` levantan `NotImplementedError` a propósito: el formato de archivo `.model` no tiene dónde guardar el `byte_shuffle`. Para volcar el vocabulario y mirarlo hay un método aparte, `save_vocab()`.

## Paso 4 — Tokens especiales (opcional)

**Qué:** que tu tokenizer maneje tokens especiales y siga coincidiendo con `tiktoken` cuando están presentes.

**Dónde:** `mi_tokenizer/regex.py`, el método `encode` (`register_special_tokens` ya viene resuelto). `GPT4Tokenizer` lo hereda. **Autochequeo continuo:** `pytest -m paso4 -v` (2 tests).

**Cómo funciona:** los tokens especiales **no pasan por BPE**. Se hace un *carve-out* previo: partís el texto por las ocurrencias exactas de cada token especial (con `re.split` y un grupo de captura para que los delimitadores queden en el resultado), emitís directamente el id de cada especial, y corrés BPE solo sobre los pedazos que quedan en el medio.

```
def register_special_tokens(self, special_tokens):
    self.special_tokens = special_tokens # str -> int
    self.inverse_special_tokens = {v: k for k, v in special_tokens.items()}
```

Los cinco especiales de GPT-4:

```
GPT4_SPECIAL_TOKENS = {
    '<|endoftext|>': 100257,
    '<|fim_prefix|>': 100258,
    '<|fim_middle|>': 100259,
    '<|fim_suffix|>': 100260,
    '<|endofprompt|>': 100276,
}
```

**El footgun de `allowed_special`.** Es el punto que vimos en clase: si tokenizás texto que viene de un usuario y aceptás tokens especiales sin pensarlo, cualquiera puede inyectar un `<|endoftext|>` en tu prompt. Por eso tanto `tiktoken` como la implementación de referencia **fallan por defecto** cuando encuentran un especial en el texto, y hay que declarar la intención explícitamente:

```
def encode(self, text, allowed_special="none_raise"):
    # "all"          -> todos los especiales se reconocen
    # "none"         -> ninguno; se tokenizan como texto común
    # "none_raise"   -> ninguno, y si aparece uno, error (default)
    # set(...)       -> solo los de ese conjunto
```

### Autochequeo:

```
texto = "<|endoftext|>hello world"

enc.encode(texto, allowed_special="all")    # [100257, 15339, 1917]
mio.encode(texto, allowed_special="all")    # [100257, 15339, 1917]

enc.encode(texto)    # tiktoken levanta ValueError
mio.encode(texto)    # el tuyo también tiene que fallar
```

Registrá **los cinco** especiales, no solo `<|endoftext|>`: los tests del repositorio usan un texto que los ejercita a todos.

Si además entrenás un tokenizer propio y le agregás especiales, acordate de que los ids de los especiales van **después** del último merge. Con `vocab_size=32768`, el último merge tiene id `32767`, así que el primer especial va en `32768`.

## Paso 5 — SentencePiece / Llama 2 (stretch)

⚠ Leé esto antes de empezar

**Este paso NO tiene solución de referencia.** Ni en este repositorio ni en la rama `solucion`: la implementación de referencia es enteramente byte-level, no hay nada de SentencePiece ni de Llama, `sentencepiece` no está en `requirements.txt`, y no hay ningún test que cubra esto.

Es decir: los Pasos 1 a 4 los podés autochequear contra código que funciona; **el Paso 5 no**. Está incluido por completitud y como disparador, no como un ejercicio con respuesta verificable.

**Qué sería:** la mayoría de los LLM que no son de OpenAI (Llama, Mistral) usan `SentencePiece` en lugar de la BPE byte-level de GPT. La diferencia principal es que **SentencePiece corre BPE directamente sobre code points de Unicode**, no sobre bytes UTF-8. Los bytes aparecen solo como fallback, para code points raros que quedaron fuera del vocabulario (`byte_fallback`).

### Si querés intentarlo:

- Reescribí tu BPE para que la unidad base sean code points (`list(text)` en vez de `list(text.encode("utf-8"))`), y pensá qué hacés con los code points que no viste en el entrenamiento.
- Explorá `sentencepiece` directamente y mirá cómo se configura un modelo tipo Llama 2 (`model_type="bpe"`, `byte_fallback=True`, normalización, el manejo del espacio con `_`,

`add_dummy_prefix`, etc.). Vas a necesitar `pip install sentencepiece`, que **no** está en el `requirements.txt` del repositorio.

- Como referencia externa, el tokenizer de Llama 2 está publicado en HuggingFace.

Tomalo como exploración libre. Si te trabás, no es que se te esté escapando algo: no hay una solución al lado con la cual compararte.

## 5. Cómo autochequearse

Tenés cuatro herramientas, de la más barata a la más fuerte:

**1. Round-trip exacto.** El chequeo más básico y el que más bugs agarra:

```
assert mi_tokenizer.decode(mi_tokenizer.encode(s)) == s
```

Corrélo sobre el string vacío, un solo carácter, texto con emoji y coreano, y el archivo `tests/taylorswift.txt` completo.

**2. El ejemplo canónico.** `"aaabdaaabc"` con 3 merges → `[258, 100, 258, 97, 99]`. Vale para los Pasos 1 y 2.

**3. Comparación byte a byte contra tiktoken.** Es el estándar de oro para el Paso 3. No alcanza con que "se parezca": las listas de ids tienen que ser **idénticas**.

**4. La suite de tests.** Es la herramienta principal, y ya está apuntando a tu código:

`tests/test_mi_tokenizer.py`, 21 tests, agrupados por paso.

```
pytest -m paso1 -v      # 5 tests
pytest -m paso2 -v      # 6 tests
pytest -m paso3 -v      # 8 tests
pytest -m paso4 -v      # 2 tests
pytest -v .              # los 21
```

Los tres primeros chequeos de esta lista ya están adentro de esos tests: hacen round-trip, corren el ejemplo canónico y comparan contra `tiktoken`. Están parametrizados sobre una lista `test_strings` que incluye el string vacío, un carácter suelto, un string con unicode y emoji, y el archivo completo — un buen punto de partida si querés agregar casos propios.

## 6. Criterio de "terminado"

| Paso | Se considera hecho cuando...                                                                                                                                                                                                                                                                                                                                                                 |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | <code>train("aaabdaaabc", 259)</code> seguido de <code>encode</code> da exactamente <code>[258, 100, 258, 97, 99]</code> ; el round-trip es exacto sobre string vacío, un carácter, emoji y el archivo completo; y entrenando con <code>vocab_size=512</code> sobre <code>tests/taylorswift.txt</code> los merges son fragmentos reconocibles del inglés y la compresión ronda <b>2,3X</b> . |
| 2    | Todo lo del Paso 1 sigue valiendo con la clase nueva; y entrenando con <code>vocab_size=512</code> sobre el mismo corpus, desaparecen los merges del tipo <code>'the ', 'e ', ', 20'</code> (contra ~110 que había en el Paso 1), quedando solo los casos que el patrón permite a propósito (espacio inicial, contracciones, puntuación + newline).                                          |
| 3    | Para los cuatro strings de prueba, <code>mi.encode(s) == tiktoken.encode(s)</code> <b>elemento por elemento</b> , y <code>mi.decode(mi.encode(s)) == s</code> . Incluí obligatoriamente un caso con no-ASCII y emoji, y el archivo <code>tests/taylorswift.txt</code> entero.                                                                                                                |
| 4    | <code>`encode("&lt;</code>                                                                                                                                                                                                                                                                                                                                                                   |
| 5    | No hay criterio verificable. Consideralo hecho si podés explicar con precisión en qué difiere el esquema de SentencePiece del byte-level BPE de GPT, y qué implica esa diferencia para idiomas no ingleses.                                                                                                                                                                                  |

## 7. Errores comunes

**encode por frecuencia en vez de por prioridad de merge.** Es el error más frecuente y el más difícil de ver, porque el round-trip **sigue dando bien**. En `encode` no importa cuántas veces aparece un par en el texto de entrada: importa **cuál se creó primero durante el entrenamiento**, o sea el menor valor en `merges`. Se implementa así:

```
pair = min(stats, key=lambda p: self.merges.get(p, float("inf")))
if pair not in self.merges:
    break    # ya no queda nada por fusionar
```

Ese `break` es necesario: si ningún par está en `merges`, todos valen `inf` y `min` devuelve uno arbitrario, con lo cual entrarías en loop infinito.

**Usar `re` en lugar de `regex`.** El patrón de GPT-4 usa `\p{L}`, `\p{N}` y cuantificadores posesivos, que el módulo `re` de la stdlib no soporta. Tiene que ser `import regex as re`.

**Olvidarse de `errors="replace"` en `decode`.** Un token individual puede ser un fragmento de una secuencia UTF-8 incompleta (medio emoji, por ejemplo). Si decodificás una lista arbitraria de ids sin `errors="replace"`, `decode` puede tirar `UnicodeDecodeError`. La convención acá es reemplazar por `?`.

**No aplicar la permutación de bytes de GPT-4.** Y aplicarla en **un solo** sentido, que es peor: te queda un `encode` que coincide y un `decode` que devuelve basura, o al revés.

**Leer archivos sin especificar UTF-8.** En Windows, `open(path)` usa la codificación local (cp1252), no UTF-8. Con `tests/taylorswift.txt`, que tiene caracteres no-ASCII, eso te rompe todo de una forma difícil de diagnosticar. Siempre:

```
open("tests/taylorswift.txt", "r", encoding="utf-8")
```

**Probar solo con ASCII.** Es la causa raíz de la mitad de los bugs que no aparecen hasta el final. Meté emoji y coreano desde el primer test.

**Intentar reconstruir los merges de GPT-4 "a mano".** No se puede a partir del vocabulario final; hace falta el mini-BPE de `recover_merges`. Copiala.

**Confundir `.model` con `.vocab`.** El `.vocab` es una salida de lectura humana, lossy (usa `errors="replace"`), y **nunca** se puede volver a cargar. El archivo real es el `.model`.

**Borrar los docstrings de los esqueletos apenas empezás.** Tienen la guía de implementación de cada método. Reemplazá el `raise NotImplementedError`, no el docstring.

## 8. Tiempo estimado y dificultad

Estimaciones para alguien que ya vio la teoría en clase y programa en Python con soltura. Si nunca trabajaste con `bytes` en Python, sumale un rato al Paso 1.

| Paso                            | Dificultad        | Tiempo estimado    |
|---------------------------------|-------------------|--------------------|
| 1 — <code>BasicTokenizer</code> | Baja-media        | 1 a 2 horas        |
| 2 — <code>RegexTokenizer</code> | Media             | 45 min a 1,5 horas |
| 3 — Reproducir GPT-4            | <b>Alta</b>       | 2 a 4 horas        |
| 4 — Tokens especiales           | Media             | 45 min a 1,5 horas |
| 5 — SentencePiece / Llama       | Muy alta, abierta | Indefinido         |

**Total Pasos 1–4: aproximadamente 5 a 9 horas.**

El Paso 3 concentra la dificultad. Si te trabás más de una hora ahí, usá el código que está en este mismo enunciado: está explícitamente autorizado copiar `recover_merges`, y el `byte_shuffle` son cuatro líneas. Lo valioso del Paso 3 no es reinventar esas dos cosas, es entender **por qué** hacen falta.

Sumale unos minutos de setup: la instalación de dependencias y la primera descarga del vocabulario de `cl100k_base`.

## 9. La solución de referencia

Existe una solución completa y funcionando, con los 21 tests en verde. **No está en la rama en la que trabajás.** Vive en la rama `solucion`:

```
git switch solucion    # ver la solución
git switch master      # volver a tu trabajo
```

Está deliberadamente a un comando de distancia: para consultarla tenés que decidir activamente hacerlo, no te la cruzás por accidente.

| Archivo en la rama <b>solucion</b>      | Qué resuelve                                                                                                                  |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <code>mi_tokenizer/base.py</code>       | Clase <b>Tokenizer</b> base: <code>get_stats</code> , <code>merge</code> , <code>save/load</code> , <code>_build_vocab</code> |
| <code>mi_tokenizer/basic.py</code>      | <b>Paso 1</b> completo                                                                                                        |
| <code>mi_tokenizer/regex.py</code>      | <b>Paso 2</b> y <b>Paso 4</b> completos                                                                                       |
| <code>mi_tokenizer/gpt4.py</code>       | <b>Paso 3</b> completo ( <code>recover_merges</code> , <code>bpe</code> , <code>byte_shuffle</code> )                         |
| <code>tests/test_mi_tokenizer.py</code> | La misma suite, corriendo contra esa implementación                                                                           |
| (nada)                                  | <b>Paso 5 — no hay solución de referencia</b>                                                                                 |

### Cómo usarla sin arruinar el ejercicio:

1. **Intentá primero sin mirar.** El valor está en escribir el loop de merges vos mismo. Los archivos son cortos y están comentados a propósito para que se puedan leer; pero leerlos antes de intentar convierte el ejercicio en un ejercicio de transcripción.
2. **Usala como oráculo, no como fuente.** Cuando tu implementación no coincide con `tiktoken`, compará tu `merges` y tu `vocab` contra los de la referencia para localizar dónde divergen. Las dos ramas usan la misma estructura de archivos, así que también podés comparar el código directamente, sin cambiar de rama:

```
git diff master origin/solucion -- mi_tokenizer/basic.py
```

Va contra `origin/solucion` y no contra `solucion` a propósito: recién clonado, el repositorio todavía no tiene una rama local `solucion`, y el comando falla con `bad revision`. Una vez que hiciste `git switch solucion` al menos una vez, las dos formas funcionan igual.

Ojo: eso te muestra la solución entera del archivo. Si estás tratando de no espoilearte, quedate con la comparación de `merges/vocab` del párrafo anterior.

3. **La excepción autorizada es `recover_merges`** en el Paso 3, y ya la tenés transcrita acá mismo, así que ni siquiera hace falta cambiar de rama.
4. **Antes de cambiar de rama, guardá tu trabajo** (commiteá o `git stash`). Si no, git te va a frenar o te vas a llevar cambios sin querer de una rama a la otra.

## 10. Referencias

### Material principal

- Andrej Karpathy — *Let's build the GPT Tokenizer* (video, ~2 h): <https://www.youtube.com/watch?v=zduSFxRajkE>

- **Material de la Clase 3 · Tokenización** de la materia (bytes y UTF-8, BPE, encode/decode, regex splitting, tokens especiales).

## Herramientas

- **tiktoken** (OpenAI): <https://github.com/openai/tiktoken>
- Tiktokenizer — visualizador interactivo de tokenización: <https://tiktokenizer.vercel.app>
- **sentencepiece** (Google), para el Paso 5: <https://github.com/google/sentencepiece>

## Lecturas técnicas para el Paso 3

- Discusión sobre la recuperación de merges desde los mergeable ranks:  
<https://github.com/openai/tiktoken/issues/60>
- Explicación complementaria de por qué el árbol de merges se puede reconstruir:  
<https://github.com/karpathy/minbpe/issues/11#issuecomment-1950805306>

## Papers

- Sennrich et al. (2015), *Neural Machine Translation of Rare Words with Subword Units* — la referencia original de BPE en NLP: <https://arxiv.org/abs/1508.07909>
- Radford et al. (2019), *Language Models are Unsupervised Multitask Learners* (GPT-2) — sección 2.2, "Input Representation", donde se introduce el BPE byte-level con regex splitting.